

AdaIN StyleKV: Training-Free Style Transfer with Self-Attention KV Fusion and Adaptive Instance Normalization for Diffusion Models

Junji Jiang Zhuoliang Lv Peiwen Yang

School of Computer Science and Engineering (Cyberspace Security),
University of Electronic Science and Technology of China

{junji.jiang, zhuoliang.lv, peiwen.yang}@uestc.edu.cn

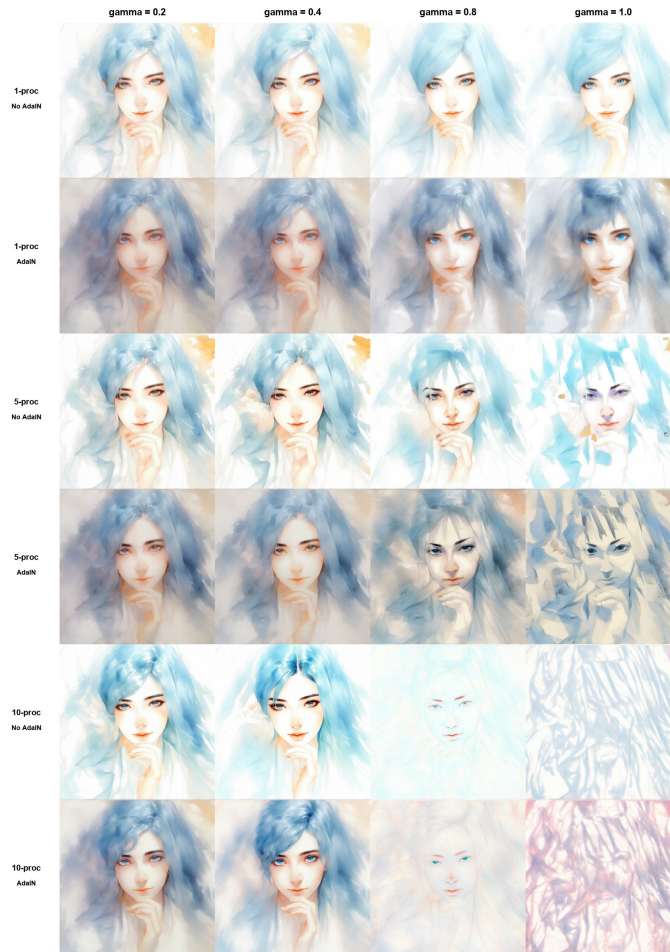


Fig. 1. Stylization results of AdaIN StyleKV. Horizontally, γ increases from 0.2 to 1.0; vertically, 1-proc, 5-proc, and 10-proc configurations are shown with and without AdaIN. The 5-proc + AdaIN configuration (our full method) achieves a favorable balance between style intensity and content fidelity near $\gamma = 0.6$.

Abstract

Artistic style transfer—taking the look and feel of one image and imposing it on another while keeping the underlying scene intact—has seen major progress thanks to diffusion models. Yet the landscape remains fragmented. Some methods need per-style fine-tuning or inversion, which is slow. Training-free alternatives, meanwhile, often trip up on one front or another: structure drifts, colors don’t cohere, or there is no easy knob for dialing style up and down. We introduce AdaIN StyleKV, a framework that sits on top of a frozen diffusion model and brings together three pieces to tackle these pain points jointly. First, a Canny ControlNet locks in edges and contours so the content image’s spatial layout stays put during generation. Second, inspired by Style Injection, we cache key-value features from a style image inside a handful of self-attention processors within the UNet block `up_blocks.0.attentions.1`, then blend them with the content KV via a single scalar γ —no gradients, no tuning. Third, we borrow the cross-modal AdaIN idea from StyleStudio [31] and adapt it to our pipeline: right at the same UNet target block, we use channel-wise statistics of a pretrained image embedding to nudge text-condition features toward the style reference’s color palette and overall atmosphere. On MS COCO (content) $\times$ WikiArt (style), a systematic sweep over 5000 image pairs shows that a 5-processor StyleKV setup without AdaIN hits the lowest Art-FID (85.43), while turning AdaIN on brings LPIPS down from 0.629 to 0.599—a measurable drop in perceptual disruption with only a modest Art-FID trade-off. We also ship a Gradio frontend, StyleKV Control Studio, where users can slide γ , tweak Canny thresholds, and mix AdaIN coefficients in real time.

Keywords: style transfer; diffusion models; self-attention; adaptive instance normalization; training-free; ControlNet

1. Introduction

Artistic style transfer has been kicking around computer vision for a while now. The setup is straightforward: you hand the algorithm a pair of images—one for content, one for style—and ask it to produce an image that reads like the first but looks like it was painted by the second. Texture, brushwork, palette all need to carry over while the scene itself stays recognizable.

The roadmap from then to now traces a clear arc. Early work stitched textures via non-parametric sampling [1]; Gatys et al. [2] recast the problem as optimization over VGG Gram matrices. Feedforward nets [3,4] and Adaptive Instance Normalization (AdaIN) [4] brought real-time speeds. GANs raised the bar on fidelity [5,6,7]. But each wave left a rough edge. GANs don’t generalize well beyond the styles they were trained on. Optimization-based methods are compute-heavy. And none of the early families gave users a straightforward slider for trading off style strength against content fidelity.

Diffusion models changed the conversation [8,9,10]. Stable Diffusion [11] in particular, with its latent-space denoising and cross-attention conditioning, made it possible to steer high-resolution synthesis from text alone. Researchers quickly repurposed this machinery for style transfer [12,13,14,15,16,17], but the results cluster around a few recurring trouble spots. InST [12] and StyleDiffusion [13] are strong but need per-image gradient steps or textual inversion—fine for a gallery, painful at scale. At the other extreme, training-free methods such as DiffStyle [14] and Style Injection [16] skip optimization entirely, yet they can warp the content’s geometry or miss the mark on global color. Self-attention is great at picking up local texture, but it is essentially blind to overall palette, so outputs can feel chromatically disjointed. And very few methods expose a clean, continuous intensity control—just a binary on/off, or weights you have to hack to adjust.

Our bet is that these problems are tangled enough that a single-component fix won’t cut it. What we need is a bundle of lightweight interventions that each patch a specific gap, working together inside a frozen model. That’s AdaIN StyleKV. It needs no fine-tuning and no textual inversion. Here’s how it started. We first built a bare-bones AdaIN prototype on SDXL with Canny ControlNet, following InstantStyle’s [17] lead on which UNet blocks matter most. AdaIN was applied on one or two target blocks to bring text and image conditions into closer alignment. Colors got better. But textures—brushstrokes, local patterns—stayed stubbornly close to the content image. That observation pushed us toward the Style Injection [16] playbook: we began caching key–value pairs from the style image and injecting them during generation. Rather than a wholesale KV swap, we use a weighted blend, controlled by a single coefficient $\gamma \in [0, 1]$. And instead of spraying KV fusion across the whole UNet, we confine it to a handful of self-attention processors inside

the single block `up_blocks.0.attentions.1`. That’s the “StyleKV” half.

The “AdaIN” half tackles what KV fusion alone misses. We adapt the cross-modal AdaIN formulation from StyleStudio [31] and apply it at the same target UNet block. A pretrained image encoder (IP-Adapter’s, in our implementation) extracts a style embedding; we then use its channel-wise mean and variance to re-center text-condition features. The idea is simple: text tells the model what to generate, but AdaIN tells it what color mood to use. Together with Canny ControlNet—which supplies an edge map as an explicit spatial anchor—the three components (structure, texture, color) form a complete, training-free pipeline.

We evaluate on 5000 COCO [18] $\times$ WikiArt [19] pairs across seven variant configurations. Art-FID [20] captures style-domain proximity, FID [21] anchors the content side, and LPIPS [22] measures per-pair perceptual drift. The headline numbers: with 5 KV-fusion processors and no AdaIN, we get Art-FID = 85.43, the lowest in our sweep—strong style, but LPIPS sits at 0.629. Flipping AdaIN on nudges Art-FID up to 90.76 but drops LPIPS to 0.599, a clear sign that AdaIN tempers the aggressive texture transplant and keeps the image feeling more coherent. Beyond the numbers, we dissect how each knob—block count, processor count, γ , AdaIN on/off—shapes the output. And to make all this tangible, we built StyleKV Control Studio, a Gradio interface where you can upload images, slide parameters, and watch the Canny edge map and stylized result update in real time.

2. Related Work

2.1. Neural Style Transfer

Gatys et al. [2] kicked off neural style transfer by matching Gram matrices of VGG [23] features inside an optimization loop. What followed was a sprint toward real-time speeds. Johnson et al. [24] trained feedforward transformers with perceptual losses. Huang & Belongie [4] gave the community AdaIN—a clean, channel-wise statistics trick that enabled arbitrary style transfer in a single forward pass. Attention-based methods [25], Transformers [26], reversible flows [27], and dual-branch learning schemes [28] each added their own spin. The common thread: speed and generalization improved dramatically relative to Gatys’ original optimizer. The common limitation: when the target style is heavily abstract or non-photorealistic, these VGG-centric pipelines start to smear fine details and flatten textures.

2.2. GAN-Based Style Transfer

CycleGAN [5] showed that cycle-consistency losses could drive unpaired image translation, and StarGAN [6] scaled the idea to multiple domains inside one model. StyleGAN [7] and its descendants [29,30] gave the field an exquisitely controllable latent space. The catch: StyleGAN was never designed for cross-domain translation. It generates, it doesn’t translate. Getting it to do style transfer on an arbitrary painting means bolting on extra fine-tuning or disentanglement machinery, and even then the results can be brittle outside the training distribution.

2.3. Diffusion-Based Style Transfer

Diffusion models have become the dominant paradigm for high-quality image generation. Among them, latent diffusion models (LDM) [11] achieve efficient training and inference by compressing images into a low-dimensional latent space. For style transfer tasks, researchers have proposed various diffusion-based methods:

InST [12] uses textual inversion to embed a style image into the text token space. Fidelity is impressive, but every new style triggers its own inversion run. StyleDiffusion [13] adds CLIP-based losses and a fine-tuning stage to push content and style apart in feature space—again, per-style optimization. DiffStyle [14] operates in the diffusion H-space and needs no training, but applying it naively to Stable Diffusion can shift semantic layout in ways that break the content. Style Injection [16] noticed that you can directly overwrite the K and V in self-attention with style-image features. That’s training-free, fast, and preserves a fair amount of structure, but without an explicit spatial anchor the content can still wander. InstantStyle [17] made the key observation that not all UNet blocks contribute equally; injecting style into a small subset is cheaper and more effective than broadcasting everywhere. StyleStudio [31] showed you can selectively gate which style elements get transferred when a text prompt is in the loop.

Table 1. Comparison of our method with representative style transfer methods. TF: Training-Free (no fine-tuning); SP: Structure Preservation (explicit spatial constraints); CT: Color Transfer (global color handling); CI: Controllable Intensity (explicit continuous control).

Method	TF	SP	CT	CI
InST [12]	×	×	✓	×
StyleDiffusion [13]	×	×	✓	✓
DiffStyle [14]	✓	×	×	×
Style Injection [16]	✓	×	×	×
InstantStyle [17]	✓	×	×	×
AdaIN StyleKV (Ours)	✓	✓	✓	✓

Table 1 places our work against these baselines. What sets AdaIN StyleKV apart is the combination: entirely training-free, a Canny edge condition to pin structure down, a cross-modal AdaIN path dedicated to global color, and one scalar γ for continuous intensity control.

2.4. Conditional Control in Diffusion Models

ControlNet [15] introduced zero-convolution layers to bolt spatial conditions (edges, depth, pose) onto a frozen diffusion model without wrecking its pretrained weights. We use the Canny variant—it takes an edge map extracted from the content image and feeds it through a trainable copy of the UNet encoder, injecting its outputs into the skip connections and middle block of the frozen decoder. Crucially, the ControlNet must match the base model. All our experiments run on SDXL with `diffusers/controlnet-canny-sdxl-1.0`. If you were on SD1.5, you would swap in `lllyasviel/control_v11p_sd15_canny` instead. We chose SDXL for its higher native resolution and

richer semantic backbone.

3. Method

AdaIN StyleKV sits on a frozen SDXL backbone with a Canny ControlNet attached. At inference time, three lightweight mechanisms work in concert: a cross-modal AdaIN that aligns text-condition features with the style image’s color statistics, a selective KV fusion that blends style key–value pairs into a handful of self-attention processors, and the Canny edge condition that keeps the scene geometry anchored. We walk through the pipeline in five stages.

3.1. Overall Framework

Stage 1: Preprocessing. Content and style images go into the VAE encoder, producing latents z_{content} and z_{style} . A Canny edge detector (thresholds adjustable in our frontend) extracts a contour map from the content image as the ControlNet spatial condition c_{canny} .

Stage 2: DDIM Inversion. We run DDIM inversion on both latents separately to obtain complete noise trajectories $\{z_t^{\text{content}}\}$ and $\{z_t^{\text{style}}\}$ across all T timesteps. These trajectories give us something to anchor both the AdaIN modulation and the KV caching to—each timestep has a corresponding style-latent to pull K/V from, and a content-latent where the actual denoising happens.

Stage 3: Cross-Modal AdaIN. At each denoising timestep, before the UNet forward pass, we intercept the text-condition embeddings at the target block(s) and apply AdaIN using channel statistics extracted from a pretrained image embedding of the style image (IP-Adapter’s encoder, in our setup). No new adapters are trained; we just shift mean and scale of the already-computed text features toward the style embedding’s distribution.

Stage 4: KV Fusion. Target block:

`up_blocks.0.attentions.1`. During the style image’s forward pass at timestep t , we record $\{K_{\text{style}}, V_{\text{style}}\}$ from a chosen set of self-attention processors inside that block. Then, during the content image’s denoising step at the same t , for each selected processor we blend:

$$\begin{aligned} K_{\text{fused}} &= (1 - \gamma)K_{\text{content}} + \gamma K_{\text{style}}, \\ V_{\text{fused}} &= (1 - \gamma)V_{\text{content}} + \gamma V_{\text{style}}, \end{aligned} \quad \gamma \in [0, 1].$$

Non-target processors run standard self-attention—no caching, no blending. The content query Q_{content} stays untouched throughout.

Stage 5: Decoding. After T denoising steps, z_0 goes through the VAE decoder to produce the stylized output.

Fig. 2 gives the visual summary of this five-stage flow.

3.2. Selective KV Fusion

3.2.1 Self-Attention Recap

The UNet self-attention layer projects a feature map X into query, key, and value:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (1)$$

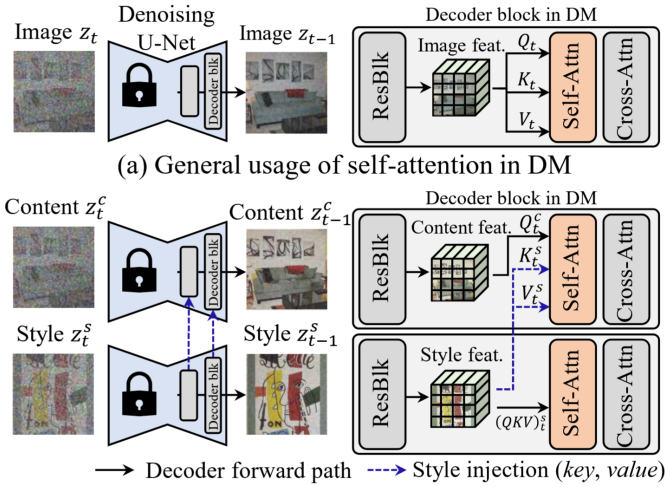


Fig. 2. Overall pipeline of AdaIN StyleKV. Content and style images enter the VAE in parallel; a Canny edge map of the content anchors ControlNet. DDIM inversion produces latent trajectories for both images. Cross-modal AdaIN fuses text and image conditions at the target UNet block, and cached style K/V are blended with content K/V inside selected self-attention processors. The VAE decoder produces the final output.

then computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

In normal generation Q, K, V all come from the same image. For style transfer the trick is to keep Q from the content image (it encodes where things are) while blending the content K, V with those from the style image (they encode what things look like—texture, brushwork, palette). The attention map computed from a content query against a style-biased key effectively “looks up” style patterns to place at content locations.

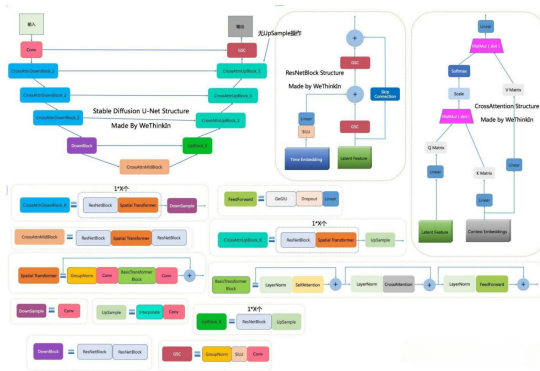


Fig. 3. U-Net architecture in Stable Diffusion, showing the encoder, middle block, and decoder with their ResNet and self-attention layers. We confine KV fusion to the target block `up_blocks.0.attentions.1` (red), leaving all other attention processors untouched.

3.2.2 Weighted KV Fusion

Following Style Injection [16], we target the self-attention processors inside `up_blocks.0.attentions.1`. But

instead of a hard replacement we use a simple linear blend:

$$K_{\text{fused}} = (1 - \gamma)K_{\text{content}} + \gamma K_{\text{style}} \quad (3)$$

$$V_{\text{fused}} = (1 - \gamma)V_{\text{content}} + \gamma V_{\text{style}} \quad (4)$$

with $\gamma \in [0, 1]$. Three things to note. First, it’s a slider: $\gamma = 0$ is pure content, $\gamma = 1$ pure style, everything in between interpolates smoothly. Second, it costs no gradient steps—just two vector additions and a scalar multiply per processor. Third, blending is gentler than overwriting; we saw fewer sudden artifacts compared to hard KV replacement in early experiments.

3.2.3 Which Block, Which Processors?

InstantStyle [17] taught us that not every UNet block pulls its weight for style. Following their lead, we fix a single decoder block as our injection site: `up_blocks.0.attentions.1`. All the “StyleKV” experiments in this paper operate exclusively there. Within that block we vary how many of its self-attention processors participate in KV fusion:

- **1 proc**: minimal injection, mostly content
- **5 procs** (our default): the sweet spot in our experiments
- **10 procs**: heavier injection, stronger style but can eat into fine detail

Five processors gave us the most even trade-off between style transfer and content preservation, so that’s the default in the frontend and throughout the paper unless stated otherwise.

Table 2. Target block configurations in the early naive AdaIN prototype. `1blk/2blk` indicate the number of UNet target blocks where AdaIN is applied; in contrast, StyleKV experiments are fixed at `up_blocks.0.attentions.1` and vary the number of self-attention processors.

Config	AdaIN Target UNet Block(s)	Variant
1blk	<code>up0.attn1</code>	V1: <code>ada_in_1blk</code>
2blk	<code>up0.attn1; down2.attn1</code>	V2: <code>ada_in_2blk</code>

3.3. Cross-Modal AdaIN for Text and Image Conditions

KV fusion handles texture. Color is a different beast. We found that blending K/V pairs does a decent job with local brushwork but often misses the global palette—you get the right strokes in the wrong hues. The StyleStudio [31] paper pointed out that AdaIN across text and image modalities can close this gap, and we adapt that insight directly into our pipeline.

The setup is simple. A frozen image encoder (the one shipped with IP-Adapter, in our implementation) produces a style embedding f_{style} . At the same target UNet block where KV fusion happens, we grab the text-condition features f_{text} that are already in the pipeline and re-normalize them:

$$f_{\text{adain}} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \mu(f_{\text{style}}) \quad (5)$$

with $\mu(\cdot)$, $\sigma(\cdot)$ computed per channel. In code we expose two extra knobs, α and β :

$$f_{\text{adain}}^{(\beta)} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \beta \mu(f_{\text{style}}) \quad (6)$$

$$f_{\text{out}} = (1 - \alpha)f_{\text{text}} + \alpha f_{\text{adain}}^{(\beta)}. \quad (7)$$

At $\alpha=1.0$, $\beta=1.0$ (our defaults) this collapses to Eq. 5. What’s happening under the hood: the text embedding’s semantic direction stays intact, but its channel-wise statistics get pulled toward the style reference. The result is that the diffusion model receives the same “what to draw” signal, but the “what color mood” signal now comes from the painting. No extra training, no new adapters—just a mean-and-variance shift at the right place in the network.

3.4. ControlNet Integration

Canny ControlNet gives us a spatial scaffold. It takes the content image’s edge map, pushes it through a trainable copy of the UNet encoder (frozen at initialization via zero-convolutions), and injects the outputs into the frozen decoder’s skip connections and middle block. We use the SDXL-matched weights `diffusers/controlnet-canny-sdxl-1.0` as-is, with no further training.

3.5. Inference

Everything above happens at inference time inside a completely frozen ϵ_θ . At timestep t , with text prompt c and Canny condition c_{canny} , the noise prediction is:

$$\hat{\epsilon} = \epsilon_\theta(z_t, t, c, c_{\text{canny}}) \quad (8)$$

The AdaIN modulation and KV fusion are injected as side-effects during the forward pass through the target block; the model weights never move. The output follows the five-stage pipeline described earlier.

4. Experiments and Results

4.1. Experimental Setup

4.1.1 Datasets

Content dataset: We use 5000 content images from the MS COCO 2017 dataset [18], each accompanied by a corresponding text prompt to assist the diffusion model in understanding the image content.

Style dataset: We use 5000 style images from the WikiArt dataset [19], covering various artistic styles such as Impressionism, Cubism, and Abstract art.

Image pairing: 5000 content images and 5000 style images are paired one-to-one in sequential order, yielding 5000 pairs in total.

Data scale: A total of 5000 stylized images are generated for quantitative evaluation.

4.1.2 Evaluation Metrics

We track three numbers, all in the lower-is-better direction, but they capture different things. **Art-FID** [20] measures distributional distance from the target style set—our primary gauge of how well the style came through. **FID** [21] does the same against the content set, giving a rough proxy for content-domain proximity (though it’s too global to serve as a standalone structure-preservation score). **LPIPS** [22] computes per-pair perceptual distance between each output and its content image. A low LPIPS means the image “reads” similar to the original; a high LPIPS means the transfer visibly shifted things. Since any interesting stylization will push LPIPS up somewhat, we read it alongside Art-FID and FID, not in isolation. No single metric tells the full story here—the three-way trade-off is the point.

4.1.3 Hardware Configuration and Hyperparameters

Experiments were conducted on the AutoDL platform using a rented NVIDIA V100-32GB GPU. Hardware and hyperparameter configurations are shown in Table 3 and Table 4, respectively.

Quantitative Comparison Across Evaluation Metrics

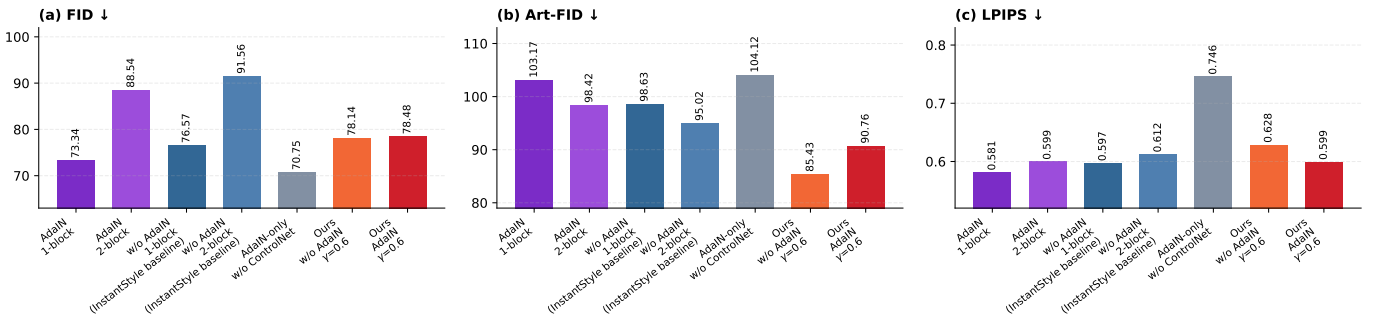


Fig. 6. Quantitative comparison of 7 method variants across FID, Art-FID, and LPIPS metrics (lower is better for all three). FID reflects distribution distance from the content domain, Art-FID reflects proximity to the target artistic style domain, and LPIPS reflects the perceptual-level image change magnitude. V6 (Ours w/o AdaIN) achieves the lowest Art-FID, indicating stronger style injection; compared with V6, the full method V7 (Ours AdaIN, $\gamma = 0.6$) reduces LPIPS, indicating milder perceptual change.

Table 3. Hardware configuration and environment.

Configuration Item	Parameter
GPU	NVIDIA V100 32GB × 1
Acceleration Library	xformers
Sampling Steps	50 steps (DDIM)
Resolution	512×512 (center crop)
Total Generated Images	5000

Table 4. Hyperparameter settings. $\gamma = 0.6$ is the experimentally determined favorable trade-off point (see Section 4.4 for sensitivity analysis); KV fusion defaults to 5 self-attention processors in `up_blocks.0.attentions.1`; AdaIN fusion intensity defaults to 1.0.

Parameter	Value	Description
γ	0.6	Style fusion intensity
<code>up0.attn1.procs</code>	5	KV fusion processor count
<code>adain_alpha</code>	1.0	AdaIN fusion intensity
<code>adain_beta</code>	1.0	AdaIN fusion bias
CFG scale	7.5	Classifier-free guidance
DDIM steps	50	Diffusion sampling steps

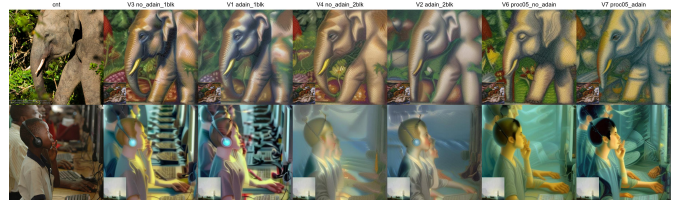
4.2. Method Variant Design

We evaluate seven configurations (Table 5) that isolate different pieces of the pipeline. V1–V5 correspond to the early AdaIN-only prototype: V1/V2 turn AdaIN on at 1 or 2 target blocks, V3/V4 turn it off (same block counts), and V5 removes ControlNet while keeping AdaIN. V6 and V7 add 5-processor StyleKV fusion on top; V6 keeps AdaIN off, V7 is the full stack. A quick note on terminology: we reserve “ablation” for single-factor comparisons—V6 vs. V7 isolates AdaIN’s effect, V1 vs. V5 isolates ControlNet. The 1blk/2blk and 1/5/10-proc variations are configuration sweeps and sensitivity analyses, not ablations in the strict sense.

Table 5. Design of 7 method variants. CtrlN = ControlNet; Scope = target blocks (blk) or processors (p); KV = KV fusion enabled.

#	Variant	CtrlN	Scope	AdaIN	γ	KV
V1	<code>adain_1blk</code>	✓	1blk	✓	—	×
V2	<code>adain_2blk</code>	✓	2blk	✓	—	×
V3	<code>noadain_1blk</code>	✓	1blk	×	—	×
V4	<code>noadain_2blk</code>	✓	2blk	×	—	×
V5	<code>adain_no_ctrl</code>	×	1blk	✓	—	×
V6	<code>proc05_no_adain</code>	✓	5p	×	0.6	✓
V7	<code>proc05_adain</code>	✓	5p	✓	0.6	✓

Fig. 4 and Fig. 5 present visual comparisons across different content–style pairings for intuitive observation of the differences among variants in style injection intensity, color coordination, and content preservation.

**Fig. 4.** Horizontal comparison of outputs from different method configurations on the “man” content image. The upper labels indicate the corresponding method configuration, enabling intuitive comparison of each configuration’s effect on style injection intensity and content preservation.**Fig. 5.** Method variant comparison on two different content–style pairings.

4.3. Quantitative Results

Table 6 presents the quantitative evaluation results of the 7 variants on 5000 generated images.

Table 6. Quantitative evaluation results of 7 variants on 5000 generated images. FID measures distribution distance from the content domain (lower indicates closer overall feature distribution to original content); Art-FID measures distribution distance from the style domain (lower indicates higher style similarity); LPIPS measures the degree of perceptual change (stylization inevitably introduces perceptual changes; moderate LPIPS values indicate effective style transfer while content remains recognizable). Bold indicates the full method (V7).

Variant Name	FID↓	Art-FID↓	LPIPS
V1 (<code>adain_1blk</code>)	73.34	103.17	0.581
V2 (<code>adain_2blk</code>)	88.54	98.42	0.599
V3 (<code>no_adain_1blk</code>)	76.57	98.63	0.597
V4 (<code>no_adain_2blk</code>)	91.56	95.02	0.612
V5 (<code>adain_no_ctrl</code>)	70.75	104.12	0.746
V6 (<code>proc05_no_adain</code>)	78.14	85.43	0.629
V7 (<code>proc05_adain</code>)	78.48	90.76	0.599

What the numbers say. V6—StyleKV with 5 processors, AdaIN off—hits the best Art-FID in our sweep (85.43). That’s the strongest style injection we measured, but LPIPS climbs to 0.629: the output looks more like the painting, but it’s also drifted further from the photo. Turning AdaIN on (V7) pushes Art-FID up to 90.76 while bringing LPIPS down to 0.599. In plain terms, AdaIN dials back the most aggressive texture replacement and compensates with better color alignment, so the image feels more coherent even though the raw style-distribution match is slightly weaker.

The AdaIN-only baselines (V1, V2) tell a complementary story. They do clean up colors but can’t get Art-FID below 98—KV fusion is what drives texture transfer; AdaIN alone can’t carry that weight. V5 (ControlNet removed) underscores the structural role: FID drops to 70.75, the best in the table, but LPIPS spikes to 0.746. Without the edge scaffold, the model is happy to match the content distribution in aggregate while warping local geometry. Doubling AdaIN blocks (V1→V2, V3→V4) intensifies the condition modulation but also nudges content structure further from the original—more isn’t always better.

Fig. 7 converts the three metrics to ranks (1 = best). The pattern is clear: no variant wins all three. V5 (no ControlNet) tops FID but trails on Art-FID and LPIPS. V6 (no AdaIN) tops Art-FID but lags on LPIPS. V7 (full) sits in the middle on Art-FID and FID while leading on LPIPS—exactly what you would expect from a method that trades a bit of raw style injection for perceptual stability.

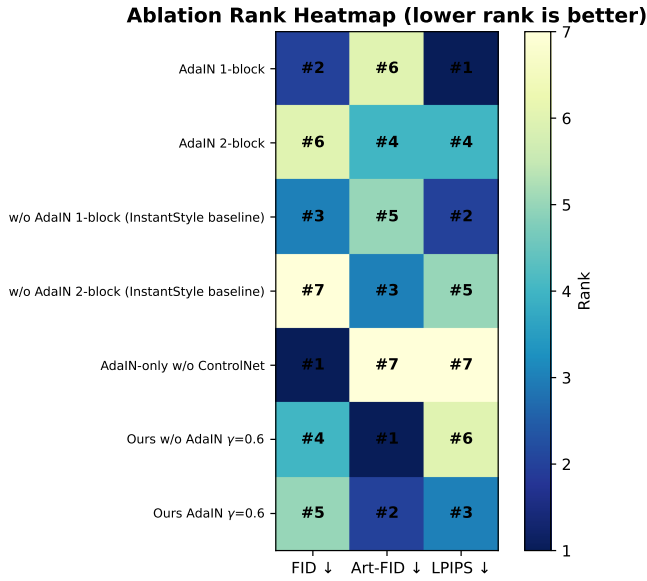


Fig. 7. Ranking heatmap of the 7 method variants across FID, Art-FID, and LPIPS metrics (lower rank indicates better performance).

Fig. 4 and Fig. 8 display visual comparison results of the variants.

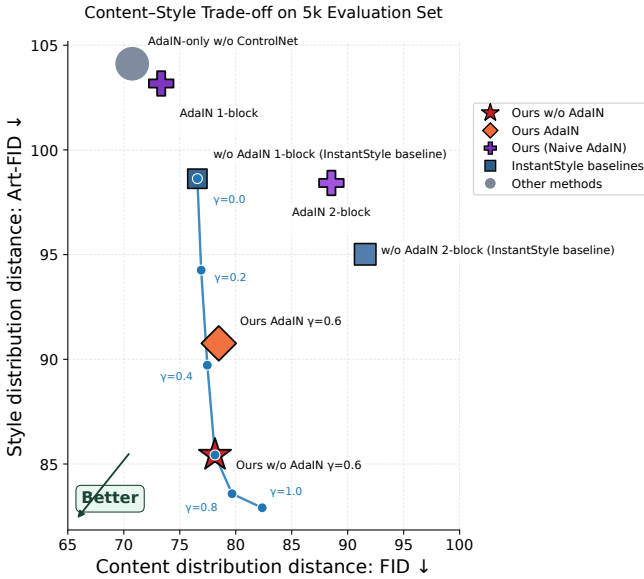


Fig. 8. Content–style trade-off comparison of method variants on FID and Art-FID.

4.4. Gamma Intensity Coefficient Sensitivity Analysis

To analyze the impact of the style fusion intensity coefficient γ on generation results, we fix the configuration of 5 self-attention processors in `up_blocks.0.attentions.1` with AdaIN and ControlNet, and vary only the value of γ , testing $\gamma \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. Experimental results are shown in Table 7.

Table 7. Results of γ intensity coefficient sensitivity analysis (5-processor configuration in `up_blocks.0.attentions.1`, fixed AdaIN + ControlNet). $\gamma = 0$ mainly retains content features with weak style injection; $\gamma = 1$ yields the highest stylization degree but with more noticeable content change. Bold indicates our default configuration, not necessarily the single-metric optimum.

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697

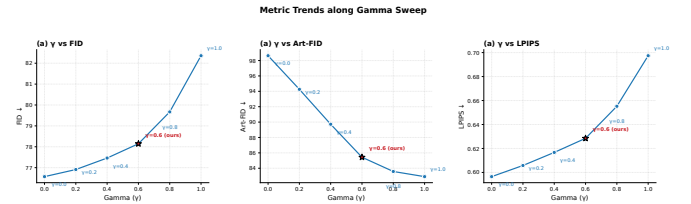


Fig. 9. Impact of γ on FID, Art-FID, and LPIPS. As γ increases, style fidelity generally improves but content change also increases; $\gamma = 0.6$ is our adopted default trade-off point.

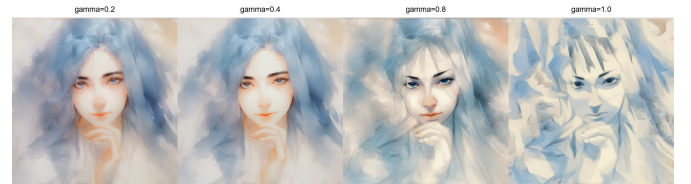


Fig. 10. Style transfer results at different γ values under the AdaIN-enabled 5-processor configuration in `up_blocks.0.attentions.1` (our method). From left to right: $\gamma = 0.2, 0.4, 0.8, 1.0$, clearly illustrating the gradual enhancement of style texture as γ increases.

Conclusion: As γ gradually increases, style texture and color expression are generally enhanced, and generated results become closer to the target artistic style distribution; at the same time, content fidelity and fine-grained structure are more susceptible to degradation. Specifically, smaller γ favors content preservation, while larger γ favors style injection. Combining quantitative metrics with visual results in Fig. 9 and Fig. 10, we select $\gamma = 0.6$ as the default trade-off point that leans toward content stability.

4.5. Qualitative Analysis of StyleKV Target Processor Count

Table 2 has already explained that `1blk/2blk` belong to target block configurations in the early naive AdaIN prototype, whereas experiments prefixed with `proc` concern the number of processors within StyleKV. These are not the same ablation dimension, so this section does not compare them as a unified quantitative ablation group. To observe the impact of the number of processors within StyleKV on style injection intensity and content preservation, we fix the target block to `up_blocks.0.attentions.1` and vary only the number of self-attention processors participating in K/V fusion within that block for qualitative visual analysis.



Fig. 11. Comparison of style transfer results under AdaIN-enabled 1-processor and 5-processor configurations in `up_blocks.0.attentions.1` at different γ values. Top row: 1-proc StyleKV configuration; bottom row: 5-proc StyleKV configuration (our method). More target processors yield stronger style injection (more pronounced style texture in the bottom row) while still maintaining good content structure in the shown examples.

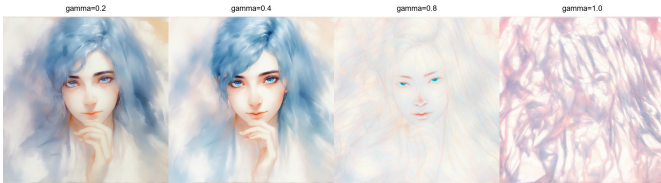


Fig. 12. Style transfer results under the AdaIN-enabled 10-processor configuration in `up_blocks.0.attentions.1` at different γ values. Compared with the 5-proc configuration in the same block, 10-proc yields stronger style injection at the same γ , but content detail preservation is slightly inferior to the 5-proc configuration.

Observation: Fig. 11 shows that within the same target block, the 5-proc configuration yields more pronounced texture transfer compared to the 1-proc configuration, especially with more sufficient style features at higher γ values. Fig. 12 further indicates that the 10-proc configuration continues to enhance stylization but is more likely to degrade facial details and local structure at high γ . Considering visual quality and computational cost together, we adopt 5 processors in `up_blocks.0.attentions.1` as the default StyleKV configuration.

4.6. Qualitative Result Analysis

4.6.1 Comparison of Different Block/Processor Configurations and AdaIN Effects

Table 8 compares the generation results of four naive AdaIN baseline configurations. Fig. 13 further illustrates the impact of the AdaIN module on color coordination under the 1-block configuration.

Table 8. Qualitative comparison of different target block counts and AdaIN configurations. Configurations using AdaIN exhibit better color coordination and more uniform style coverage; configurations without AdaIN show stronger style transfer but more noticeable content structure degradation.

Variant	#Blocks	Observation
adain_1blk	1	Concentrated style injection, noticeable texture transfer
adain_2blk	2	Uniform style coverage, better color coordination
no_adain_1blk	1	Strong style transfer, slightly inferior content preservation
no_adain_2blk	2	Enhanced style injection, more blurred structure



Fig. 13. Impact of the AdaIN module on style transfer (1-block configuration). Top row: results with AdaIN; bottom row: results without AdaIN. The AdaIN module significantly improves color coordination and overall atmospheric consistency, with the difference being more pronounced at higher γ values.

Fig. 1 (the teaser at the beginning of the paper) presents the overall transfer results of our method in a comprehensive comparison.

4.6.2 Impact of Prompts

In qualitative experiments, we observe that prompt quality significantly affects generation results. When using simple prompts, the model has limited content understanding and lacks detail; when using detailed prompts, the model can more accurately capture user intent, and style consistency and visual quality are generally better. This suggests that in style transfer tasks, text prompts are an important factor in guiding the diffusion model to understand content semantics.

4.7. Key Experimental Findings

4.7.1 Trade-off Between Canny Edge Map Resolution and Inference Time

The granularity of the Canny edge map directly affects generation quality and inference efficiency. Table 9 tests inference times at different Canny resolutions.

Key findings: (1) Finer Canny edge maps improve generation quality, as the model receives richer structural information; (2) however, excessively fine edge maps may cause the model to over-respond to local edges, producing outlining artifacts, fragmented textures, or unnatural local details; (3) inference time increases markedly with Canny

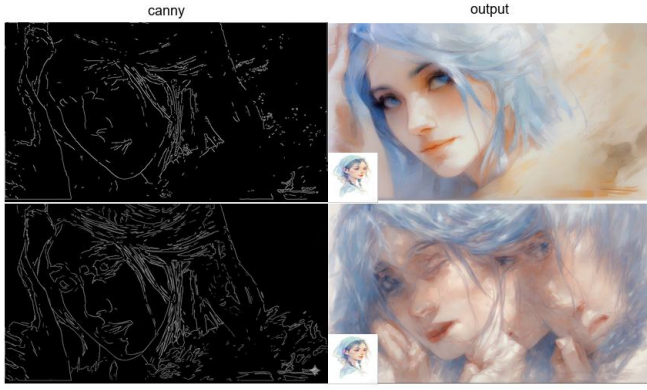


Fig. 14. Comparison of structural conditions and generation results at different Canny resolutions. Left column: Canny edge maps; right column: corresponding stylized outputs, with the reference style image embedded in the bottom-left corner of each output.

Table 9. Canny resolution vs. inference time. Higher resolution provides richer structure but increases latency and may introduce artifacts.

Resolution	Canny Size	Time
1080×599	same as input	~12 s
5774×1536	same as input	~68 s

resolution; (4) a balance must be struck between too fine (many artifacts, high cost) and too coarse (loss of detail). As shown in Fig. 14, the difference in Canny edge detection granularity across resolutions is significant. Based on these observations, the frontend supports dynamic adjustment of Canny edge detection parameters by the user.

4.7.2 Device Resource Constraints

Diffusion models (Stable Diffusion base model ~7–8 GB, plus VAE, ControlNet, and feature caching modules, totaling ~15–20 GB of VRAM) still pose certain challenges for consumer-grade GPUs. Even with a V100-32 GB, batch generation faces VRAM pressure and GPU resource contention. Compared with performing injection across more blocks or processors, our approach adopts selective processor injection within `up_blocks.0.attentions.1` (5 self-attention processors by default), thereby controlling the additional computation and caching overhead so that the entire pipeline can run smoothly on a V100.

5. Interactive Frontend

To enhance practical usability and user experience, we built an interactive frontend called **StyleKV Control Studio** based on the Gradio framework.

5.1. Frontend Functional Modules

(1) Image Input Module: content image upload, style image upload, automatic generation and preview of the Canny edge map.

(2) Prompt Control Module: positive prompt (describing the desired generation result), negative prompt (elements to avoid), with support for long text input.

(3) ControlNet/Canny Parameter Adjustment:

Canny Low/High Threshold (edge detection sensitivity), Conditioning Scale (ControlNet control strength), Bilateral Filter parameters (edge preprocessing denoising).

(4) AdaIN Style Fusion Module: AdaIN Alpha (fusion intensity), AdaIN Beta (fusion bias), with real-time preview support.

(5) Generation Parameter Control: Steps (DDIM sampling steps), CFG Scale (classifier-free guidance), Seed (random seed), KV fusion intensity γ .

(6) Output Management: output directory setting, real-time generation status display, result gallery showcase.

5.2. Frontend Features and Value

The frontend supports hot parameter reloading (adjusting parameters without restarting to view results), real-time Canny edge map preview, saving configurations as JSON files, and batch generation. The design goal is to lower the usage barrier, enabling non-expert users to understand the impact of each parameter on generation results through intuitive sliders, switches, and preview images. Fig. 15 shows the overall interface layout of the frontend.

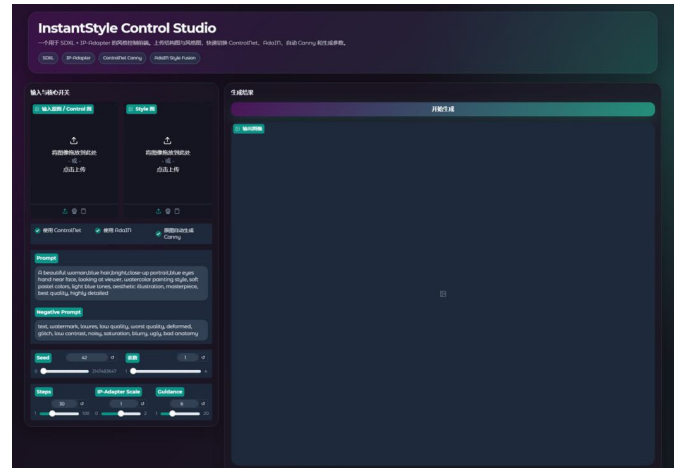


Fig. 15. Interface of the StyleKV Control Studio interactive frontend. Through the left panel, users can upload content and style images, adjust ControlNet/Canny parameters, set AdaIN fusion coefficients, and configure generation parameters; the right panel displays real-time Canny edge map previews and stylized output results.

6. Future Work

6.1. Quantitative Comparison with Existing Methods

Our current experiments focus on self-ablation to verify the effectiveness of each component and have not yet been compared with methods such as InST [12], StyleDiffusion [13], and InstantStyle [17] under equivalent conditions. We plan to conduct systematic inter-method comparisons under a unified dataset and evaluation protocol.

6.2. Adaptive Canny Edge Map Generation

Currently, Canny thresholds must be manually adjusted, and optimal values vary depending on image content. We plan

to introduce better edge extraction models such as HED and RINDNet to realize adaptive edge map generation, reducing the need for manual parameter tuning.

6.3. VLM-Based Automatic Prompt Generation

Currently, prompts need to be manually written or generated with the help of external models. We plan to introduce open-source vision-language models (e.g., Qwen-VLM, LLaVA) to automatically generate high-quality prompts for style images, further lowering the usage barrier.

6.4. Cross-Architecture Adaptation and Lightweighting

Our method is validated on SDXL, and its effective attention processor selection and fusion strategy is architecture-dependent. We plan to adapt the KV fusion mechanism to other diffusion architectures such as SD1.5 and SD3, re-identify the effective self-attention processors under each architecture, and explore lightweighting schemes such as distillation and quantization to reduce dependence on high-VRAM devices.

6.5. Incorporating Tile ControlNet

High-resolution Canny edge maps may introduce artifacts. We plan to use Tile ControlNet to mitigate artifact issues at high resolution through patch-wise processing while maintaining detail richness.

6.6. Processor-Level Contribution Analysis

Currently, we only test naive AdaIN configurations with 1 or 2 target blocks, and StyleKV configurations with 1, 5, or 10 self-attention processors in `up_blocks.0.attentions.1`. We plan to individually test the style transfer contribution of different self-attention processors in U-Net, establishing a processor-to-style mapping to provide a theoretical basis for method optimization.

6.7. Multi-Condition ControlNet Fusion

Currently, only Canny ControlNet is integrated. We plan to explore the synergistic operation of multiple ControlNets (edge + depth + pose, etc.) to further enhance content preservation capability.

7. Conclusion and Limitations

7.1. Conclusion

We put forward AdaIN StyleKV, a training-free style transfer pipeline that combines three lightweight mechanisms inside a frozen SDXL+ControlNet backbone: (1) selective KV fusion—cache style K/V from a handful of self-attention processors in `up_blocks.0.attentions.1` and blend them with content K/V via a single γ ; (2) cross-modal AdaIN, adapted from StyleStudio [31], to pull text-condition features toward the style image’s color statistics; and (3) Canny ControlNet for spatial anchoring. On 5000 COCO×WikiArt pairs, the full configuration lands at Art-FID 90.76 with LPIPS 0.599, while the AdaIN-free StyleKV variant pushes

Art-FID down to 85.43 at the cost of higher perceptual disruption (LPIPS 0.629). We also ship a Gradio frontend that exposes every knob to the user in real time.

7.2. Limitations

Although our method achieves favorable results, the following limitations remain, as listed in Table 10. These limitations also constitute the key focus areas of the future work outlined in Section 6.

Table 10. Major limitations of our method and corresponding improvement directions.

Limitation	Description	Improvement
No external SOTA comparison	Only self-variant ablation; not compared with InST, StyleDiffusion, InstantStyle, etc.	Reproduce and compare under unified protocol
Insufficient processor ablation	Only tested few processor configs in <code>up0.attn1</code>	Full processor-level contribution analysis
Single ControlNet	Only Canny tested; other modalities not explored	Multi-condition ControlNet fusion
External prompt dependency	Prompts from external models; no integrated VLM	Integrate Qwen-VLM / LLaVA
Limited evaluation metrics	FID, Art-FID, LPIPS only; no perceptual/semantic coverage	Add metrics + human subjective study
High device requirements	Needs V100-class GPU; not consumer-friendly	Distillation & quantization

7.3. Future Outlook

Style transfer is far from solved. As diffusion models keep evolving, a few directions strike us as especially worth pursuing: pushing training-free methods toward finer style disentanglement (so users can independently control brush scale, palette, and texture density); fusing multiple conditions (text, reference image, color palette) without them fighting each other; and mapping out which UNet layers actually matter for style, so future methods can be leaner by design. Getting these pipelines to run on consumer hardware—through distillation or quantization—would also go a long way toward making them useful outside the lab.

Acknowledgments

We are grateful to the AutoDL platform for GPU resources and to the open-source community behind Stable Diffusion, ControlNet, and the diffusers library—without these foundations this project wouldn’t have been possible.

References

- [1] A. A. Efros and T. K. Leung, “Texture synthesis by non-parametric sampling,” in *Proc. ICCV*, 1999.
- [2] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image style

- transfer using convolutional neural networks,” in *Proc. CVPR*, 2016.
- [3] D. Ulyanov, V. Lebedev, A. Vedaldi, and V. Lempitsky, “Texture networks: Feed-forward synthesis of textures and stylized images,” in *Proc. ICML*, 2016.
- [4] X. Huang and S. Belongie, “Arbitrary style transfer in real-time with adaptive instance normalization,” in *Proc. ICCV*, 2017.
- [5] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” in *Proc. ICCV*, 2017.
- [6] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, “StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation,” in *Proc. CVPR*, 2018.
- [7] T. Karras, S. Laine, and T. Aila, “A style-based generator architecture for generative adversarial networks,” in *Proc. CVPR*, 2019.
- [8] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proc. ICML*, 2015.
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proc. NeurIPS*, 2020.
- [10] Y. Song and S. Ermon, “Improved techniques for training score-based generative models,” in *Proc. NeurIPS*, 2020.
- [11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. CVPR*, 2022.
- [12] Y. Zhang, N. Huang, F. Tang, H. Huang, C. Ma, W. Dong, and C. Xu, “InST: Inversion-based style transfer with diffusion models,” in *Proc. CVPR*, 2023.
- [13] Z. Wang, L. Zhao, W. Xing, and D. Lu, “StyleDiffusion: Controllable disentangled style transfer via diffusion models,” in *Proc. ICCV*, 2023.
- [14] J. Jeong, M. Kwon, and Y. Uh, “DiffStyle: Training-free style transfer with diffusion models,” *arXiv preprint arXiv:2312.01714*, 2023.
- [15] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” in *Proc. ICCV*, 2023.
- [16] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, and D. Cohen-Or, “Style Injection: Training-free style transfer using self-attention,” *arXiv preprint arXiv:2312.08008*, 2023.
- [17] H. Wang, Q. Wang, X. Bai, Z. Qin, and A. Chen, “InstantStyle: Free lunch towards style-preserving in text-to-image generation,” *arXiv preprint arXiv:2404.02733*, 2024.
- [18] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014.
- [19] WikiArt, “WikiArt Visual Art Encyclopedia.” [Online]. Available: <https://www.wikiart.org/>.
- [20] M. Wright and B. Ommer, “ArtFID: Quantitative evaluation of neural style transfer,” in *Proc. GCPR*, 2022, arXiv:2207.12280.
- [21] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in *Proc. NeurIPS*, 2017.
- [22] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proc. CVPR*, 2018.
- [23] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. ICLR*, 2015.
- [24] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual losses for real-time style transfer and super-resolution,” in *Proc. ECCV*, 2016.
- [25] S. Liu, T. Lin, D. He, F. Li, M. Wang, X. Li, Z. Sun, Q. Li, and E. Ding, “AdaAttN: Revisit attention mechanism in arbitrary neural style transfer,” in *Proc. ICCV*, 2021.
- [26] Y. Deng, F. Tang, W. Dong, C. Ma, X. Pan, L. Wang, and C. Xu, “StyTr²: Image style transfer with transformers,” in *Proc. CVPR*, 2022.
- [27] J. An, S. Huang, Y. Song, D. Dou, W. Liu, and J. Luo, “ArtFlow: Unbiased image style transfer via reversible neural flows,” in *Proc. CVPR*, 2021.
- [28] H. Chen, L. Zhao, Z. Wang, H. Zhang, Z. Zuo, A. Li, W. Xing, and D. Lu, “DualAST: Dual style-learning networks for artistic style transfer,” in *Proc. CVPR*, 2021.
- [29] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of StyleGAN,” in *Proc. CVPR*, 2020.
- [30] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, and T. Aila, “Alias-free generative adversarial networks,” in *Proc. NeurIPS*, 2021.
- [31] M. Lei, X. Song, B. Zhu, H. Wang, and C. Zhang, “StyleStudio: Text-driven style transfer with selective control of style elements,” in *Proc. CVPR*, 2025.

Appendix

A. Complete Data of the 7 Variants

Table 11 lists the complete experimental data of the 7 variants, consistent with Table 5 in the main text.

Table 11. Complete quantitative data of the 7 variants (consistent with Table 5 in the main text).

Variant Name	FID↓	Art-FID↓	LPIPS
V1 (adain_1blk)	73.34	103.17	0.581
V2 (adain_2blk)	88.54	98.42	0.599
V3 (no_adain_1blk)	76.57	98.63	0.597
V4 (no_adain_2blk)	91.56	95.02	0.612
V5 (adain_no_ctrl)	70.75	104.12	0.746
V6 (proc05_no_adain)	78.14	85.43	0.629
V7 (proc05_adain)	78.48	90.76	0.599

B. Complete Gamma Ablation Experiment Data

Table 12 lists the complete data of the γ ablation experiment.

Table 12. Complete γ ablation experiment data (5-processor configuration in `up_blocks.0.attentions.1` + AdaIN + ControlNet, consistent with Table 6 in the main text).

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697